



Configuring and Managing Kubernetes Security

Securing Cluster Access

Introducing Authentication and RBAC

Securing access to your Kubernetes cluster is the foundation of Defense in Depth. You need to control who can access the Kubernetes API and what they can do with your resources, otherwise every other security measure can be bypassed. We'll start with authorization. Kubernetes uses role-based access control to manage permissions. You define what access is available, whether you can delete pods or read config maps. Those go into a set of rules, and you also define who the rules apply to. That all lives in your application model with your other YAML specs. There are four resource types you need to know. Roles define a set of allowed permissions. A role contains rules that specify actions as verbs, get, list, create, delete. Those are combined with resources, pods, services, secrets. You might create a role called pod-reader that allows getting and listing pods, but nothing else. Roles are namespace scoped, they express access levels to objects inside the same namespace where the role is deployed. RoleBindings connect roles to security principals, either service accounts or users. A RoleBinding says this principal gets the permissions from this Role in this namespace. The binding is what actually grants the permission. You can have multiple bindings to the same role, granting the same permissions to different accounts and multiple bindings per account, giving principals several sets of permissions. ClusterRoles work like Roles, but have two purposes, they can define permissions for cluster-wide resources like nodes or namespaces themselves, they can also serve as templates for permissions that can be used across multiple namespaces. Kubernetes includes default ClusterRoles like view, edit, and admin that provide standard permission sets. There's also cluster-admin, which grants everything on every resource, useful for break-glass scenarios when you need to fix something quick, but dangerous for everyday use. ClusterRoleBinding's grant ClusterRole permissions across the entire cluster. Bind the view ClusterRole to a monitoring service account, it can read resources in every namespace. Bind cluster-admin to a user, they become root for your entire cluster. You can also use a RoleBinding with a ClusterRole, you could give a user view access to the test namespace, but they may have admin for their own namespace. Every API request gets evaluated against these RBAC rules, it could be an app inside the cluster making REST calls to the Kubernetes API server or from a remote user running kubectl commands. Kubernetes identifies the authenticated caller, the security principal. Next, it finds all relevant RoleBindings and ClusterRoleBindings, and



combines the permissions. Then it makes an allow or deny decision, no permissions means no access. Wired Brain Coffee failed their latest security audit because they ignored RBAC. Service accounts were running with admin access because someone set that up for debugging and never removed it. End users with cluster-admin roles, because that's the easiest thing to set up, no way to expire accounts, so once someone is given access, they always have access, those are classic security anti-patterns that attackers exploit daily, and it's easy to address them.

Demo: RBAC for Service Accounts

Let's see what happens when teams ignore RBAC. We'll look at a typical scenario at Wired Brain Coffee, an application deployed without considering security. If you want to follow along with the demo, you just need access to a Kubernetes cluster, and you can run this setup script to deploy the initial resources. First, we check what's already in the cluster. We have the standard namespaces, and in the default namespace, a debugging pod that someone left running, and the default service account that exists in every namespace. Now, we deploy the kube-explorer application, a web interface for managing Kubernetes resources. The Wired Brain SRE team want to use this so they can stop giving kubectl access to users. The deployment spec is very simple, it just defines the container, port, and a config setting. Then, there's a service resource which exposes the HTTP port of the matching pod on port 8019. We'll deploy this with kubectl apply, and then check the pod. We're querying to get the name of the service account, which is the security principle the pod will use if the app connects to the Kubernetes API. We didn't specify that, so it's using the existing default account. The application starts up, and we browse to localhost port 8019. It shows pods in the default namespace, we can view and delete the debug pods. Refresh, and the debug pod has been recreated by its deployment object. This seems reasonable, the team wanted an app to manage pods. Here's where it gets dangerous. Switch to the kube-system namespace in the UI. The application still works, we can see all the critical system pods, the components that keep the cluster running. Worse, we can delete them, try deleting a system pod, and it disappears. Kubernetes recreates it, but the delete succeeded. Our simple pod manager has cluster-wide admin access. How did this happen? Let's check what permissions the default service account actually has. The kubectl get clusterrolebinding command with this query shows all the roles the default service account has, and someone created a clusterrolebinding, giving it cluster-admin permissions. This is disturbingly common, during debugging someone grants cluster-admin to make things work, planning to fix it later, or they assume the service account is safe because no one else will use it. But this is the default account, so any pod which doesn't have an explicit service account set will use it and becomes a cluster-admin. Time to fix this. The solution starts with creating a dedicated service account for the application, never use the default service



account for anything that needs API access. The RBAC YAML defines three things, first, a Role called default-pod-admin with specific permissions, get, list, and delete on pods, but only in the default namespace, second, a new ServiceAccount called kube-explorer, third, a RoleBinding that connects the ServiceAccount to the Role. The updated deployment specifies the ServiceAccount explicitly, that's the only change. When we apply this, Kubernetes rolls out new pods using the correct ServiceAccount. The `kubectl get pod` command confirms the pods now use the kube-explorer ServiceAccount, not default. The RoleBindings query shows the new service account just has the new pod administrator role in the default namespace. Testing the restricted application, shows exactly what we want. Browse to the default namespace, everything still works. We can view and delete pods here because that's what the role allows. Try to view service accounts forbidden, the role doesn't include service accounts, so no access. The real test is kube-system. Browse there, and we get 403 Forbidden. The role only exists in the default namespace, so the service account has zero permissions anywhere else, this is namespace isolation working as designed. We can verify these permissions with `kubectl auth can-i`. The kube-explorer service account can get and delete pods in default, but cannot access pods in kube-system. Now, the team has a new requirement, they need to monitor pods in kube-system, but shouldn't be able to delete them. We create additional RBAC resources, a new Role kube-system with only get and list permissions, no delete, and a RoleBinding connecting our existing service account to this new Role. After applying the update, we can see both rolebindings. The service account has different permissions in different namespaces, full pod management in default, read only in kube-system. The application picks up the new permissions immediately. No restart needed, browse to kube-system, and we can now see the system pods. But try to delete one, and it's forbidden. We have read access, but not write, exactly as configured. RBAC applies the authorization policies for an authenticated user. Service accounts authenticate using a token, which the API server creates and injects into the pod's file system. Here there's the CA certificate and the token. This is a JSON Web Token containing the serviceaccount identity. Every API request includes this token, and Kubernetes evaluates the current RBAC rules to determine what's allowed. JWTs are encoded, and we can paste it into jwt.io to read the contents. We can see it's a token issued by Kubernetes, identifying the subject as the service account. This is a long-lived token which is valid for a year, but you can configure that to have much shorter tokens as the default. The transformation from cluster-admin on the default account to least privilege on a dedicated account is a huge security improvement. Instead of every pod having root access, each application gets only what it needs. A compromised container can only affect resources explicitly granted through RBAC.

Understanding End-user Authentication



You need to have a good understanding of RBAC, because the default permission set up in Kubernetes isn't necessarily secure. We've seen how the Wired Brain Team accidentally gave every pod cluster-admin access by misconfiguring the default service account, then fixed it with dedicated accounts and least privilege RBAC. Service accounts secure access from inside the cluster when applications need to talk to the API using a token to authenticate. But most applications don't need API access at all, application pods mostly just serve their function, they don't manage in cluster resources, so they don't need to authenticate. For these workloads, you should disable ServiceAccount token mounting entirely, set automountServiceAccountToken to false on the Pod spec, or better still on the ServiceAccount. No token means no API access, even if someone compromises the container. End user access, humans connecting with kubectl, is different. Kubernetes doesn't manage users internally, there's no User object to create, instead, it trusts external systems to vouch for identity through certificates, tokens, or custom authentication, and then RBAC takes over for authorization. Certificate-based authentication is what you get by default. The cluster CA signs a certificate with the username in the common name field, simple, works everywhere, but certificates can't be revoked individually, and there's no multi-factor authentication. Once someone has that certificate, they have access until it expires, which might be years. Certificates, or static tokens, get embedded in the config file, which kubectl uses for cluster connections, so it's basically a plain text password file. If that file gets compromised, then an attacker has all the access your legitimate user should have. And the kube/config file has all your cluster connection details. If admins can quickly switch between clusters with no additional authentication for production, it's very easy to make a mistake, you think you're connected to the test cluster when you delete the database, but actually you've just wiped all your production data. Wired Brain Coffee's audit found exactly these problems. Using the default Kubernetes authentication with certificates valid for a year, kubeconfig files left on jump boxes for easy connection to clusters, no MFA anywhere. Again, classic symptoms of a team who are focused on delivery and not security. But Kubernetes has integration points for authenticating with external identity providers, and these issues are simple to fix.

Demo: MFA and RBAC for End-users

Let's fix Wired Brain Coffee's end user-access problems. We'll transform their AKS cluster from using static credentials to Azure Entra integration with enforced MFA and role-based permissions. We've already got a cluster set up with some other prerequisites, an AD group for users who should have cluster-admin permissions, and a separate user account who should have minimal access to the cluster. There's a setup script here if you want to try this yourself. Starting with a default aks cluster, we get credentials using the az command line. The kubectl get nodes command works, no additional authentication required, examining the



kube config with kubectl config view shows how that works. Getting the credentials embeds a token and certificate, kubectl doesn't show it, but it's in the file, encoded, but not encrypted. Anyone who gets this kube config has cluster access indefinitely. Time to enable proper authentication. First, we identify which Azure Entra group should have admin access. The az ad group commands find our cluster-admins group and get its object id. The az aks update command transforms authentication. We enable Azure Entra integration with `-enable-aad`, specify the admin group, and `-disable-local-accounts`, that last flag eliminates the backdoor. Now those certificates and tokens in kube config files don't get access anymore. The update takes a few minutes as Azure reconfigures the authentication backend. Checking in the Azure Portal, we can see that the cluster is now set up to use Entra ID for authentication and standard Kubernetes RBAC for authorization. Now, with Azure integration, when we fetch credentials again, the experience is different. The get nodes command fails with an error about a tool called kubelogin. The cluster now requires proper authentication through Azure Entra, and kubelogin handles the OAuth flow between kubectl and Azure. Kubelogin is a standard install, download the binary from GitHub, extract it to a place in your path, and make it executable. This tool is the integration between kubectl and Azure Entra. Other cloud providers have similar tools, they handle token acquisition, refresh, and caching. The updated kube config looks completely different. Instead of embedded credentials, we see an exec command that calls kubelogin, the parameters identify my account, and when I complete the login, that returns a token. The token includes my authenticated user id and the groups I belong to, that gets sent by kubectl with all API requests. Running kubectl get nodes now triggers real authentication. The tool displays a device code and URL. Opening that URL, starts the Azure Entra login flow. I'm already signed in, so I don't have to keep re-authenticating, I just need to confirm that I trust this application. The authentication happens in the browser, where Azure Entra can enforce all your security policies, conditional access, device compliance, and location restrictions. After successful authentication, kubectl commands work normally. The token gets cached locally, so you don't re-authenticate for every command, but it has a short expiry, typically one hour, forcing periodic reauthentication, this limits exposure if a device is compromised. Testing permissions with kubectl auth can-i shows full access. Because my user is in the ad group, we configured as cluster admins in the AKS update. The auth can-i list command shows I have permissions to all actions on all resources, this is excessive for most users. Now, we configure access for a developer with minimal permissions. The least-privileged YAML creates a RoleBinding to grant the built-in viewer role to this named Azure Entra user in the Wired Brain namespace. This will let the user debug their own apps, check on service endpoints, and view pod logs, but nothing else. To test this properly, we create the RoleBinding, then clear my Azure session, and sign out from the Portal. This simulates a different user accessing the cluster. Authenticating as the least-privileged user triggers the same MFA flow. My organization has MFA configured, we need both password and a second



factor, in this case, it's a phone notification, which I can complete off screen. These are the Azure Entra rules you set in your organization, separate from Kubernetes and applied to all users. Getting credentials with a different kubeconfig file name keeps context separate, preventing accidental use of admin credentials. Running `kubectl get nodes` with the custom config file triggers the device code authentication. I'm already logged in as the least-privileged user, so I can select that account. After authentication, I've just got the minimal permission set. Trying `kubectl get nodes`, returns forbidden. The user is authenticated, but is not authorized for that operation. They can only get pods in the Wired Brain namespace, exactly as configured. I can't see pods in the system namespace. And when I list all my permissions, I only see get verbs. This demo uses Azure and AKS, but the pattern is the same elsewhere. EKS with AWS IAM, GKE with Google Cloud Identity, or any cluster with OIDC and Keycloak, they each have their own integration with `kubectl`, so the principle is the same, eliminate static credentials, enforce MFA through your identity provider, use temporary tokens, and apply least privilege through RBAC.

Focusing on Defense-in-depth

We've locked down cluster access for Wired Brain Coffee with MFA for end users, tied to the same RBAC modeling approach. Kubernetes supports OpenID Connect integration, which means you can delegate authentication with pretty much any identity provider. Wired Brain run on AKS so they use Azure Entra ID and enforce security policies organization-wide. Authentication and authorization are separate. You can model the security requirements for applications and users with roles and bindings, which live alongside your application spec. The team who owns the app decides on the access levels. Authentication for end users is handled outside of the cluster by your identity provider, that means you can enforce whatever policies your organization needs, MFA as a typical minimum. When `kubectl` sends the token onto the Kubernetes API server, it contains the authenticated user's identity and groups, that's used to find the roles and evaluate the permissions. Defense in Depth is a popular buzz phrase, but it's a really important mindset to get into. Does the application you're running even need access to the Kubernetes API? Every pod in your cluster is a potential attack vector. There might be a supply-chain vulnerability hidden in a dependency, or you may be using an outdated image with known security issues. Compromises do happen and then RBAC becomes your containment system. The attacker can't jump namespaces, can't read secrets they shouldn't access, can't delete critical workloads, each permission boundary forces them to work harder, making more noise and increasing your chances of detecting them. Securing end-user access is critical because users have legitimate reasons to be powerful, they debug production issues, deploy updates, investigate problems, but they may also get into bad habits, sharing credentials for convenience, fall for phishing attacks, MFA and temporary tokens mean a stolen



laptop or leaked kube config is unlikely to give an attacker any access. And for production systems, you can use temporary elevation with an approval process, so users get admin permissions only when they need it. Many teams stop here thinking access control is enough, but can you trust the applications running inside the container? Protecting the Kubernetes API blocks a powerful attack vector, but if your apps get compromised, an attacker still has an entry point to your systems. The next layer in Defense in Depth is controlling what gets deployed in the first place, container scanning with security policies that prevent vulnerable or misconfigured workloads from running.

Securing Application Deployment

Understanding Supply-chain Complexity

Container images are packaged with the complete software stack your application needs, but you need to manage that stack to keep it secured. Every container image is an entire software supply chain, which you don't control. Your Python application starts with a base image from Docker Hub, that builds on Debian or Alpine Linux, and that base image contains hundreds of packages maintained by different teams on independent release schedules. You add your framework, Django, Flask, Fast API, each with their own dependencies, then your application requirements, which pull in their requirements, creating a tree of code from dozens of sources. The Dockerfile hides a huge collection of release schedules. Debian and Python have predictable schedules with releases yearly or biannually. Frameworks and libraries release on their own schedule when they have new features or bug fixes. Each component follows its own timeline and can have emergency patches for a security fix, any changes could break your application. This challenge gets worse the longer you leave it. You scan your image when you deploy and find it's clean. For the next deployment, you scan again, and a critical CVE is discovered in a dependency. Your application is now vulnerable. You can update the library, but maybe that breaks compatibility with your framework. You can update the framework, but maybe that needs a newer version of Python. You update Python, and now half your other dependencies don't work. In my Plural sight course, Docker for Software Development: Deploying Apps, I show you how to use Trivy to scan images. Integrating that in your CI/CD pipeline is a great start, but a continuous challenge needs a continuous solution. Ongoing scanning is the key to managing this, continuous monitoring with a system component that checks all the images in your production cluster. The scanning database is regularly updated, you know when new CVEs are found, and you can patch critical issues as soon as they're discovered. You're making regular incremental updates, so it's much easier to keep all your dependencies in sync. At Wired Brain Coffee, the security audit flagged that they only scanned images on deployment. Checking all their



production images again, they found critical vulnerabilities in everyone. Now they have a lot of updates to apply and regression test because the supply chain complexity overwhelmed their manual update processes. Continuous scanning of all container images would give them an audit log of known issues and an actionable set of fixes.

Demo: Image Scanning with Trivy Operator

Let's see how Wired Brain Coffee can approach automatic enforcement with two open source tools, Trivy for scanning and Kyverno for enforcement. You can follow along with this demo using any Kubernetes cluster. I'm running Docker Desktop. We start by installing Trivy Operator, using helm, adding the project repository, and installing. I'm setting a memory allocation and specifying the ARM CPU platform because I'm running Docker on Apple Silicon. As soon as the operator is deployed, it will start scanning, checking images for all pods in all namespaces for known vulnerabilities, and producing a report on each. Now let's deploy Wired Brain's products API using their old vulnerable image. The deployment is a simple Kubernetes spec. The image tag references an old version they've been meaning to update. This image has known vulnerabilities, but when we go to deploy it, that works just fine, and the pod starts up. We don't get any errors or warnings, and this vulnerable container is now live in the cluster. But Trivy will have started the scan, we can watch for vulnerability report resources in the namespace with the pod. It will take a few minutes, so we'll skip on until it's ready. And now we have the report, let's see what Trivy found. Lots of issues, too many to make sense of. So let's query the YAML and print the summary. There are known vulnerabilities at all levels of severity, including four-critical. We'll scale the deployment to zero replicas. This removes the vulnerable pod, but it preserves the deployment and the vulnerability report for the image. If we deleted the pod entirely, Trivy would clean up the report and then need to re-scan on the next deployment. Now we transform from detection to prevention using Kyverno, which is a Kubernetes native policy engine. Installing Kyverno, adds admission-control to your cluster, so you can block resources being created if they fail policy rules. Using helm again, we deploy to the Kyverno namespace. The pods in the namespace handle different responsibilities. The admission-controller evaluates policies, the background-controller handles existing resources, and the cleanup-controller manages policy violations. Together, they create a security gate that resources must pass through and provide an audit log on cluster activity. Kyverno adds its own custom resources, and this is how you define policies with familiar Kubernetes specs. Kyverno can query any object in the Kubernetes API, including the vulnerability report custom resources, which the Trivy operator creates. This spec is for a ClusterPolicy object, it will operate on all workload resources, pods, and pod controllers like Deployments. The policy triggers when pods are to be created. Then it makes an apiCall, fetching the vulnerability report on the image



for the pod being deployed. If there are any critical vulnerabilities, the deployment gets rejected. The error message includes the CVE count so developers know the scale of the problem. Let's test this protection, we'll try deploying the same vulnerable image again. This time, the deployment fails immediately with a clear error message. The image contains four CRITICAL severity CVEs and is blocked by security policy. No pod gets created, the vulnerable image is stopped at the gate. This transforms the developer workflow before they deployed first and maybe checked security later. Now security is a deployment requirement. Developers know they have four critical CVEs to fix, and they can query the vulnerability report for details about which packages need updating. Let's deploy the patched version to see the complete flow. The secure deployment only changes the image reference to use a new version where the team have fixed the critical vulnerabilities. They updated the base image, patched the framework, and refreshed dependencies. The deployment succeeds because this is a new image with no vulnerability report. Now we'll wait for Trivy to create the report and skip on until it's ready. The new vulnerability report shows zero critical CVEs. There might be other severity issues, but the policy only blocks critical vulnerabilities. This balance keeps developers productive, while preventing serious security issues. We could scale the deployment to zero and scale up again, and this time the new pods would be created because they passed the cluster policy rules. The reports accumulate over time, creating a security inventory. You can query all vulnerability reports to see which images need attention, track improvement trends, and prove compliance during audits.

Vulnerability Reports and Policies

Image scanning is a crucial part of your security story, and tools like Trivy Operator and Kyverno are Kubernetes native. The Trivy operator scans all container images for every pod and produces a vulnerability report. There's a custom resource which stores the details of the image and the results of the scan, there's a summary report and a detailed description of each vulnerability. Because these are standard Kubernetes resources, you can automate your response to vulnerability reports. You could simply archive them to long-term storage or logs as your audit trail, or you could trigger actions like creating tickets to address a new critical vulnerability. You can also use them for policy rules. Kyverno is a policy engine which lets you use cluster policies to restrict what can run in Kubernetes. It's verbose like all policy languages, but it's defined as YAML. You can specify rules to be enforced, which means Kyverno will block any violations. In the rules, you specify the objects to monitor, like pods. Then the context to query the state of the resources, this is a Kubernetes API query, so you can look up the vulnerability report for the container image or pretty much anything else. Then the validate section specifies the conditions, like denying if there are any critical vulnerabilities. We saw how to use the Trivy operator to continuously monitor images, and Kyverno to block any pods being



created from images with critical vulnerabilities. But there are limitations, unknown workloads are permitted because Trivy can only find images to scan by looking at the running pods. If there are CVEs, you can block new pods being created, but that will break auto-scaling and updates while you still have a vulnerable container running. It's best to use vulnerability reports for ongoing detection and notification. What you should enforce in production are configuration best practices. Don't run as root. Root processes can modify system files, install packages, and potentially escape the container. Don't mount the host file system, this gives containers direct access to files on the node, including other containers data and potentially Kubernetes secrets. Drop dangerous capabilities, system feature like net admin or sys admin, grant kernel-level privileges that can bypass container isolation. These aren't vulnerability issues that change daily, they're fundamental security practices that only have valid exceptions for certain applications. Wired Brain's audit showed how security can't rely on the human factor. The security team had documented all these best practices, but development teams are focused on delivery, so they may not make time for security, that's why policies need to be enforced in the platform.

Demo: Enforcing Policies with Kyverno

Let's implement comprehensive security policies that address all the ways Wired Brain Coffees cluster security failures. If you want to follow along, you'll need access to a Kubernetes cluster and Trivy operator and Kyverno installed. The setup script does that for you. We're deploying two policy files, the first enforces two rules that work as gates, registry restrictions, and non-root requirements. Looking at the YAML, the rules have validationFailureAction set to Enforce. The restrict-image-registries rule checks that the image reference in the container spec matches our approved pattern, this only approves pods for creation if they're from the production image registry in ghcr. The require-non-root rule validates that run as non-root in the pod spec is set to true, that restricts any pods which don't specify that in the security context or don't have a security context section at all. The second policy handles CVE checking differently. ValidationFailureAction is set to Audit, not Enforce. This rule queries the Trivy vulnerability report API for the image in the pod spec and checks if critical count is greater than zero. It won't block deployments, it only logs violations. This prevents new CVEs from disrupting production operations while still providing visibility. Let's apply both policies. These are custom resource definitions which get created with kubectl, like any other object. The cluster policies apply to all pod deployments across the cluster, they're written with helpful messages that explain what the issue is. First, let's test the registry restriction. It's easy to grab images from Docker Hub, but you can't guarantee they're secured, this policy blocks that shortcut. The test-deployments uses then nginx image from Docker Hub, that's docker.io. This is an official image which we can trust, but Wired Brain want to own all the images they use, everything needs



to be in their own private registry. When we try to create this, the deployment fails with a clear message, Images must come from the approved registry. This forces teams to use the organization's images, where they have control over the base layers, the update cycle, and the vulnerability scanning. Next, the root user restriction. This deployment uses an approved registry, but doesn't set a security context, not in the pod spec or in the containers specification. The container would run as whatever user the image specifies, which is likely to be root. When we try to deploy, again, Kyverno blocks it immediately, the error tells developers exactly what's missing, Containers must run as non-root user for security. Third, let's test the CVE check. This deployment passes both enforce checks. The spec has `runAsNonRoot` set to true, and sets a user. It uses an image from the approved ghcr registry, but that image tag references the vulnerable version with critical CVEs. Apply this, and the deployment succeeds, but Kyverno still checked and has audited the issue. The deployment wasn't blocked, but the violation was recorded, this prevents CVE discoveries from breaking scaling or rollouts, while still providing security visibility. Now let's deploy something that passes all checks. This manifest meets all the policies, `runAsNonRoot` for containers, ghcr for the image registry, and this is using the fixed image with no critical vulnerabilities. The deployment is successful, and the pod starts running. Let's verify the security configuration. The `securityContext` shows `runAsNonRoot` is true with user ID 65534. The image comes from our approved registry. When Trivy completes scanning, the vulnerability report will show no critical issues. Now here's the audit trail. Kyverno automatically generates policy reports that track every policy evaluation. Here, we can see the deployments which succeeded, there's a failure flag in the image with known CVEs, but three passes for the compliant deployment. We can print the detail of the report to see the results. For the compliant resource, we see all the rules have passed. The combination of enforcement and audit provides flexibility. Critical security issues, like root containers get blocked immediately. Evolving issues, like new CVEs, get logged for visibility without disrupting operations, you get security without sacrificing availability.

Focusing on Defense-in-depth

Admission control using a policy engine is one of the strongest security vectors you can add to your Kubernetes clusters. Kyverno policies secure what can run in Kubernetes. Enforcement policies block workloads from running at all. We saw rules for enforcing an approved image registry and preventing containers running as root, you could extend that to block system capabilities or privileged containers. Other policies allow the workload to run, but report on issues for auditing, ideal for images with critical CVEs. More flexible rules are good if you know some apps can't meet them, using writeable file systems, old base images, or the latest image tag. As you harden your cluster, you create barriers attackers need to bypass. Kyverno provides admission control, blocking bad configurations



before they run. Falco is another CNCF project built for runtime security, detecting suspicious behavior when pods are running. Auditing is another layer of defense, tracking what happens in your cluster so you can analyze attacks if they do happen. Kubernetes audit logs can track every API call, revealing who did what and when. Trivy operator can publish metrics to Prometheus, showing vulnerability trends over time. This visibility enables better security without slowing delivery. The security team now has data to identify which policies need adjustment for different teams or different workloads. An image registry policy might be too strict if teams needed also use staging registries for testing. A non-root policy might need exceptions for vendor applications, those exemptions are declared in the model as labels or annotations. The audit data helps to calibrate policies that work with development workflows. When developers get clear error messages instead of silent failures, they know exactly what to fix. Automation freed Wired Brain's developers from an impossible task, they don't need to track thousands of CVEs across hundreds of dependencies because Trivy operator handles continuous scanning. They don't need to remember security best practices under deadline pressure because Kyverno enforces policies automatically, they don't need to become security experts, the infrastructure won't let them ship insecure code. On their security journey, Wired Brain have restricted access to the cluster and restricted what can run inside the cluster. The last missing piece is securing communication, making sure incoming and outgoing traffic is all restricted to trusted locations.

Securing Applications at Runtime

Securing Communication with Network Policies

Defense in Depth is the key to securing Kubernetes, and the last piece is securing applications at runtime. We've built two layers of defense for Wired Brain Coffee, RBAC and MFA control, who can access the cluster and what they can do. Image scanning and admission policies, control what gets deployed, these reduce the likelihood of compromise. But there are still attack vectors in the running application, a supply chain attack or a zero-day CVE gets discovered in an approved base image, or there's an insider threat, you can't assume there are no bad actors inside the organization. Runtime security limits the damage if there is a compromise. The main factors to secure are communication between components and the storage of sensitive configuration data, like database credentials or API keys. Kubernetes networking is open by default. Any pod in any namespace can communicate with any other pod. A compromised container is like a jump box which an attacker can use to go anywhere else in the cluster. Network policy is the solution, it's part of the standard Kubernetes API which acts as a firewall with traffic rules. Ingress specifies which pods can send



traffic in, egress specifies which pods you can send traffic out to. Traffic that doesn't match and allow rule gets blocked. Network policies work on labels. A policy identifies the pods it applies to using a selector. Pods which are allowed ingress are identified by another selector, and pods which are allowed for egress have a selector. Typically, you start with a default deny policy in each namespace that uses an empty selector to match all pods. It's a failsafe because if someone forgets to apply a policy to a pod, it's effectively outside of the firewall. Then you add specific policies permitting only the traffic your application needs. For the Wired Brain app, the database should only allow ingress from the APIs, the API should only allow egress to the database, and ingress from the web app. The web app should only allow egress to the APIs.

Demo: Network Policies as the Firewall

In this demo, we'll implement network policies as a firewall for Wired Brain's cluster. You can follow along with this demo on any Kubernetes cluster with a network implementation that supports network policy, Docker Desktop doesn't, so I'm using k3d cluster running locally. First, we'll deploy the application with no network restrictions. The app.yaml file defines the complete Wired Brain application. There are four deployments, the Postgres database, the products-api, the stock-api, and the web frontend, each has labels identifying the component, which we'll use for network policy selectors later on. We deploy with kubectl apply and check the pods are running in the wiredbrain namespace. Now we proved the problem exists, logically, the web frontend should only communicate with the APIs which then talk to the database. But with no network policies, any pod can reach any other pod. We exec into the web deployment and use netcat to test connectivity to the database. The connection succeeds. The web tier can reach the database directly, bypassing the API layer. Time to establish a zero-trust baseline. The default-deny policy uses an empty podSelector, which matches all pods in the namespace. We specify both Ingress and Egress in the policy types, but we don't define any rules, so all traffic is blocked. These are standard Kubernetes resources, we can apply the policy and describe it to confirm the configuration. The network policy shows the empty selector and the denied traffic types. Let's verify the lockdown is in place. Running the same netcat command to test database connectivity now fails because Egress from the web pod is blocked. We can also try to reach the product's API with an http get, that fails too, and it tells why. The DNS lookup fails before we even attempt the connection. Blocking all Egress means pods can't even connect to the DNS service in the cluster to resolve names. DNS is infrastructure traffic that almost everything needs. The allow-dns policy restores it by permitting Egress to pods in the kube-system namespace on port 53. The namespace selector uses the metadata name label that Kubernetes adds automatically. Network policies are additive. When we apply this, pods in the Wired Brain namespace will be able to use DNS, but not make any other network



calls. If we retry the http get to the product's API, we can see it looking up the service name, but the actual request still fails. We've allowed DNS queries, but nothing else. Now, we can selectively enable the traffic paths the application needs. The allow-api-to-db policy applies to pods with the component label set to database, and it allows traffic in from pods with components set to products API and from pods labeled stock API. The allow-api-egress policy is the other half of the rule, it selects the products-api pods and allows traffic out to the database component. Network policies work in both directions. You need egress allowed at the source and ingress allowed at the destination. There are more policies to enforce the rest of the rules, allowing ingress to the APIs from the web pods and egress from the web pods to the APIs. We can apply all the policies in the folder. Network policies are checked at runtime, just like RBAC policies, so we don't need to restart any workloads. Let's test the segmentation. First, the path that should be blocked. The netcat command from the web pod to the database still fails. There's no policy allowing that traffic, so it's denied. Now, the path that should work, the http get to the product's API endpoint succeeds. We get a JSON response listing products. The web pod can reach the API and behind the scenes, the API is reaching the database to fetch that data. Finally, we test egress to the internet, which should be blocked for all our pods because there is no policy allowing it. The http get to my blog URL is refused. Our policies only allow traffic within the cluster to specific destinations. A compromised pod can't download attack tools or exfiltrate data to external servers. This is network segmentation, working as designed. Traffic flows through the intended paths, web to API to database, everything else is blocked.

Why You Need the CSI Secrets Driver

Securing applications in Kubernetes is all about restricting what pods can do. Network policies now control traffic flow in the Wired Brain cluster. Each pod can only access its immediate downstream component. A compromised web pod can't reach the database directly or pods in any other namespace. It can't communicate outside of the cluster, so attackers can't exfiltrate data, that's solid security at the network layer. The other runtime security concern is protecting sensitive data, database credentials, API keys, and certificates. The Wired Brain team use the built-in Kubernetes secret resource, but it's not a secure option. Kubernetes secrets store data as base64-encoded strings, that's encoding, not encryption. Anyone with read access to secrets can decode them. Kubectl even has a helper method for that. The Wired Brain audit found database passwords that any developer could read, RBAC can prevent that by restricting secret access, but the data at rest is usually completely unprotected. The cluster database, etcd, can encrypt secrets at rest, but that requires additional configuration, which is often skipped. The proper solution is external secrets management. Sensitive data needs to live in a dedicated storage service outside of the cluster, that could be Azure Key Vault, AWS Secrets Manager, or



HashiCorp Vault. Applications retrieve secrets at runtime, which get loaded into pods as volumes. The Secret store CSI driver powers this integration. You include a SecretProviderClass in your model, which is an abstraction over an external store. You specify the provider and the parameters to connect, then you explicitly state the secrets to fetch. In your Pod spec, you reference that ProviderClass as a volume, then you can mount the volume in your container and read the value from the secret store as a text file. Authentication to the secret store is seamless, without any cloud credentials in your model. You use annotations to link the Kubernetes service account to a cloud security principle, Azure Entra app registration, AWS IAM role, or Google Cloud service account, that's used to authenticate to the vault. The benefits go beyond encryption, centralized secrets across clusters with a single place to manage them, versioning of secrets and rotation without having to redeploy your apps, authorization rules separate to the RBAC in your cluster with audit logging for every access.

Demo: Secure Secrets with CSI Driver

In this demo, we'll see how to use the Secret Store CSI Driver with Azure Key Vault. I've already created all the Azure components. If you want to follow along, there's a setup script in the course downloads. To start with, we're using a helm chart with standard Kubernetes Secrets. The database deployment references a secret to populate the POSTGRES_PASSWORD environment variable. The secret itself is defined with String data, which means we can use plain text for the password when we deploy. The values file sets the actual password. This is the first attack vector, credentials are stored in source control, anyone with repository access can read this file and see the database password. We deploy with helm install, creating the wiredbrain -namespace and waiting for the pods to start. Now let's see how secure those secrets really are. The kubectl gets secret command with yaml output shows the data field. We can query that with yq to see the encoded value, that's base64, not encryption. It's easy to decode, the go-template output format with base64 decode reveals the plain text password immediately. Anyone with read access to secrets in this namespace knows the database credentials, that's the second attack vector. Let's look at how the pod consumes this. The JSON query extracts the environment configuration from the database pod spec. The password is injected as an environment variable. And if we exec into the pod and run printenv, there it is in plain text. That's the third attack vector, anyone who can exec into the pod sees the credentials. The setup script created an Azure Key Vault with the secrets stored properly. The az keyvault secret list command shows we have three secrets for the database password, these are in different formats we need for different components, the secret show command retrieves the values, for the database, we just have the password to set, for the Java properties, this is a key-values pair. These are stored encrypted in Key Vault, protected by Azure RBAC, and every access is logged. Time to upgrade to the secure architecture. The new helm chart uses the



Secret Store CSI Driver instead of Kubernetes Secrets. The `SecretProviderClass` defines the Key Vault integration, one for each secret so we can explicitly map just the data each component needs. The `ProviderClass` specifies the `vaultName` and names the secret to fetch. The `objectName` matches the Key Vault secret name, this is for the database password. You can use `objectAlias` to rename the secret, and this becomes the name of the file inside the pod. The `serviceaccount` includes the workload-identity annotation, linking it to an Azure-managed identity. This identity has permission to read from Key Vault so pods using this service account can retrieve secrets without any static credentials. The deployments now use CSI volumes instead of secrets. This is the database spec. We set the `workload.identity` label so this pod has the context of the Azure principle with Key Vault access. Instead of injecting a secret as an environment variable, we point to a file path where the secret will be mounted. The volumes section adds a CSI volume using the Secrets Store Driver. Referencing the database `secretProviderClass`, we upgrade the helm release with a secure chart and additional values file that configures the CSI integration. Helm handles the transition, creating the new resources, and rolling out updated deployments. Let's verify the secrets are gone from Kubernetes. The `get secret` command for database credentials returns not found. There's no secret object anymore. And running `printenv` for the `POSTGRES_PASSWORD` variable in the database pod returns nothing. Instead, the password is mounted as a file. Listing the secrets directory, shows `postgres-password` file, reading it shows the credential. The CSI driver retrieved this from Key Vault when the pod started, and mounted it as a read-only file, which is actually stored in memory on the node. The password never gets persisted, it exists only in Key Vault and in the running pod's file system. That one attack vector remains. If someone can exec into this pod, they can read the file.

Completing the Security Model

Wired Brain Coffee will pass their next security audit because they've implemented a full security model. The final part of that is runtime containment, network policy segment traffic, so pods can only communicate where they should, compromised pods can't pivot through the cluster. External secrets with the CSI Driver mean credentials aren't stored in etcd, they get loaded from a secure external vault. Every pod only sees the secrets it needs in the file system, so if there is a breach, the blast radius is contained. There's more that can be done at the container level in the pod spec, but that needs care because it can break your application. You can apply a `seccompProfile` to Linux containers to restrict dangerous system calls. And for individual containers, you can turn off Privilege Escalation, use a `readOnlyFilesystem`, run as a `NonRoot` user, and remove all unnecessary capabilities. But you'll need to apply those one at a time and test your app because they might cause a failure. Runtime security is complemented by deployment security. Image scanning will tell you if a CVE is



found, but it's likely to be contained by the increased pod security, which can be enforced with policies in the cluster, and that deployment security is complemented by cluster security, ensuring only valid users have access and applying RBAC to limit what they can do. We've used a specific toolset at Wired Brain, Azure Entra, Kyverno, Trivy, and Key Vault, but the principles work across clouds and distributions. This is what Defense in Depth looks like, least privileged access controls for users and applications, ongoing vulnerability scanning, enforced as one of many security policies, restricted access to networks and secrets for containment if there is a compromise. Security shouldn't be seen as a checkbox, new vulnerabilities emerge and attack techniques evolve, and these are the foundations for proactive defense.